

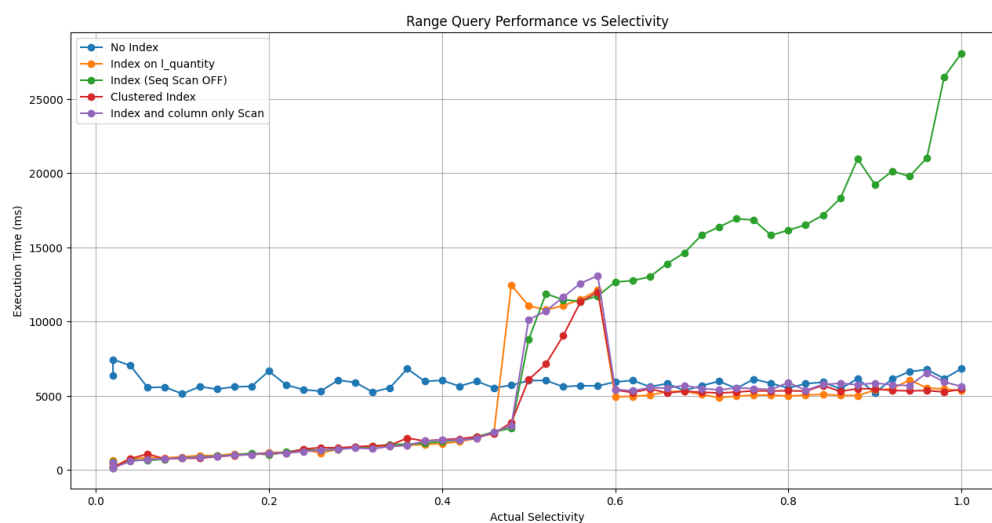
Analysis PDF for DBMS assignment 1, Q2, Part B

Introduction

This experiment evaluates the performance of range queries on the lineitem table under varying selectivity levels and different configurations of indexing. The range predicate used in all queries is: WHERE l_quantity <= X.

We use the same program developed in Part A to compute the query parameter values corresponding to the required selectivity levels. The computed query parameters are carried forward and used consistently for all experiments in this part.

Graph of Range Query Performance vs Selectivity



Axes:

X Axis: Actual Selectivity

Y Axis: Execution Time (in ms)

Execution Scenarios:

- 1) **No Index** – Sequential scan without any index
- 2) **Index on l_quantity** – We create an index and find the values of selectivity.
- 3) **Index with Sequential Scan Disabled** – Index scan forced
- 4) **Clustered Index on l_quantity** – Table clustered on index key
- 5) **Index-only Scan** – Query rewritten as SELECT l_quantity with index

Observations and Reasoning

a) No Index

Observation: The execution time remains nearly the same across all values of selectivity.

Explanation: Since there is no index in the table, SQL scans the entire table to execute the SELECT COUNT(*) query irrespective of the selectivity value. Hence, the time taken to scan does not vary significantly with the actual selectivity value.

b)Index on l_quantity

Observation: The execution time is initially very low for smaller selectivities but then significantly increases after crossing 50% selectivity and later approaches a constant value.

Explanation: At low selectivities, SQL uses index scans to scan the entire table to execute the SELECT COUNT(*) query. However, as the value of selectivity increases, the number of rows scanned too increases and eventually the index scanning does not help much. Hence, SQL eventually switches to a sequential scan and eventually the time taken to scan approaches a constant value just like case a: no index.

c)Index with sequential scan disabled

Observation: The execution time is initially very low for smaller selectivities but then significantly increases as the value of actual selectivity increases.

Explanation: Since sequential scanning is disabled, SQL is forced to use index scanning which are although efficient at small values of selectivities, but tend to become really inefficient for higher values of selectivities with increase number of records which SQL is forced to scan. After traversing through the BTree and arriving at a particular index, SQL has to fetch the tuple corresponding to the index in the BTree, which are scattered across many pages and repeated fetching can be very inefficient.

d)Clustered Index

Observation: The execution time is initially similar to the non-clustered case, but at medium and higher selectivities, the execution time tends to be lower than the corresponding non-clustered.

Explanation: Clustering physically reorders rows in the table and as selectivity increases, range queries benefit from the improved spatial locality which is favoured by clustering. This reduces the random page scanning and fetching and reduces the execution time at medium and higher selectivities.

e)Index (Column-only) Scan

Observation: The execution time is initially very low for small to medium selectivities but then increases as the value of actual selectivity increases, performing worse than the index scan case b.

Explanation: This query retrieves data only from one column (l_quantity) which is directly stored in the BTree index. Hence, all required data is already present in the index and SQL does not need to access the Heap for fetching tuple data leading to increased efficiency. However, as the selectivity increases, the number of indexes to be accessed increases and hence the increase in efficiency is not significant.

Performance metrics and timing comparisons

Execution time was measured in milliseconds (ms) using Python's `time.perf_counter()`. Each query was executed once for each value of selectivity in [0.00, 0.02, ..., 1.00] for each of the 5 main cases represented by 5-line plots in the graph above.

Justification for design choices

- Binary Search was used for calculating the value of X in part A corresponding to the given selectivity values as the selectivity is directly proportional with X and its time complexity is $O(\log n)$ making its execution possible in a given time.
- 50 selectivity points were used to get a more accurate graph to understand the difference between the 5 cases.
- Having 5-line plots on a single graph helps us compare the 5 cases easily.
- Using the `time.perf_counter()` function in python ensures high-resolution measurement.

System Configuration details

a)Hardware Configuration

OS Name: Microsoft Windows 11 Home Single Language

OS Version: 10.0.26200 N/A Build 26200

CPU Information: 13th Gen Intel(R) Core(TM) i7-13700H with 14 cores and 20 logical processors.

RAM: 16 GB

Storage: HDD, 512 GB

b)Software Configuration

Python Version: 3.12.10

PostgreSQL Version: 17.6

Database: tpch_db lineitem table

c)PostgreSQL Configuration

Number of shared buffers = 4GB

Working memory = 4MB

Maintenance working memory = 64MB

Effective cache size = 4GB

Random page cost = 4

Sequential page cost = 1

Max parallel workers per gather = 2

Planner Settings:

SET enable sequential scan = OFF;

SET enable index scan = ON;

SET enable bitmap scan = ON;